Assignment 2

CUDA Virus Signature Matching — Final Report

1. Implementation Description

1.1 Algorithm

- The matcher compares every sample against every signature to find the best alignment score.
- To avoid brute-force checking across all positions, signatures, and samples, the final version uses **anchor-based pre-filtering**: preprocessing each signature to extract a 10-character **anchor** which is the first sequence of 10 consecutive characters that are not N.
- Each anchor is then hashed into a lookup table with kAnchorKeyCount = 1 << 20 buckets, using the hash:
  h = (h * 131 + char) & (kAnchorKeyCount - 1)
- If a signature does not contain any valid 10-character non-N segment, it is stored in a separate fallback list.
- During matching, each GPU thread computes the same 10-character hash for its current sample position and checks the corresponding bucket.
- Only positions whose hash maps to a non-empty bucket continue to full character-by-character matching.
- This greatly reduces unnecessary work. For min_clean, which has 700 signatures over 1 million positions, only about **0.07% of positions became the fallback**. That means **99.93% of threads can skip** the expensive O(signature_length) verification step.

The matching works like this:

- N on either the sample or signature matches any character
- The Phred score of an N base is always 0
- Match confidence is the average Phred score over matched sample positions only
- For each (sample, signature) pair, the best match is the one with the highest confidence
- If confidence ties, the later start position is preferred
- To support both rules efficiently on the GPU, results are packed as: (score_sum + 1) << 32 | start
- This allows a single atomicMax to select both the highest-confidence match and, in the case of ties, the latest starting position.

1.2 Kernels and Grid Dimensions

Kernel	Grid shape	Block size	Role
computeIntegrityHashes	(ceil(samples/256), 1)	256	Uses one block for each sample. The 256 threads read the sequence in strides, accumulate partial Phred sums, then reduce them with warp-shuffling.

Kernel	Grid shape	Block size	Role
computeBestMatches	(ceil(max_pos/256), samples)	256	Here, blockIdx.y identifies the sample and blockIdx.x identifies the position tile. Each thread handles one sample position, performs the anchor hash lookup, and only then proceeds to full verification if needed.
computeFallbackMatches	(ceil(fallback_pairs/256), 1)	256	Uses one thread for each (sample, fallback_sig) pair and performs a brute force scan for signatures that are entirely N or do not have a usable anchor.

Block size 256 is chosen at runtime using `cudaOccupancyMaxActiveBlocksPerMultiprocessor`. Instead of just comparing how many blocks can run at once, the implementation compares the total number of active threads by checking whether `active_blocks_256 * 256 >= active_blocks_128 * 128`. This gives a fairer comparison because it better reflects the effect of register and shared memory limits on actual occupancy.

1.3 Memory Handling

Buffer	Host allocation	GPU memory region	Description
sample_bases, sample_qualities (~1 GB each)	new[]	**Global memory**	Used by computeBestMatches and computeIntegrityHashes, with accesses served through L2 cache
Signature bases (~2 MB)	new[]	**Global memory**	Read during full verification in computeBestMatches and computeFallbackMatches
Offsets, lengths, packed results	cudaHostAlloc	**Global memory**	Small metadata buffers, pinned on the host side to support asynchronous H2D and D2H transfers
Anchor table (bucket_starts, bucket_sig_indices, bucket_anchor_offsets)	cudaHostAlloc	**Global memory**	Stored in a Compressed Sparse Rows and used as a read-only lookup structure during anchor filtering. Small enough to fit well in L2 cache
Per-thread		**Registers**	Kept in registers: 28

Buffer	Host allocation	GPU memory region	Description
accumulators (hash, score_sum, loop indices)			registers per thread based on profiling
Hash kernel partial sums (cross-warp reduction)		**Shared memory**	Used only to hold the 8 warp-level totals before the final reduction, so the shared memory usage is very small at 32 bytes per block
Packed best-result array (uint64_t, samples × sigs)		**Global memory**	Updated with atomicMax by both computeBestMatches and computeFallbackMatches

2. Runtime Analysis

2.1 Baseline Performance (Final Implementation)

Case	Samples	Sigs	Sample len	Sig len	N%	runMatcher (s)	vs bench-a100-1	vs bench-a100-2
min_clean	1,000	700	1,000,000	3,000	0%	**2.68**	—	—
high_n	1,000	700	1,000,000	3,000	10%	**2.79**	—	—
more_signatures	1,000	1,000	1,000,000	3,000	0%	**2.78**	—	—
more_samples	2,200	700	1,000,000	3,000	0%	**5.55**	—	—
longer_samples	1,000	700	2,000,000	3,000	0%	**4.84**	—	—
longer_signatures	1,000	700	1,000,000	10,000	0%	**2.85**	—	—

2.2 Factor Analysis

Factor changed vs min_clean	Change	runMatcher delta	Explanation
N 0.0 → 0.1 (high_n)	+10% N chars	+0.11s (+4%)	The anchor hash is still computed in the same way, so the overall work barely changes. The main difference is that some sample windows are a little less likely to produce useful anchor matches.
Signatures 700 → 1,000 (more_signatures)	+43% sigs	+0.10s (+4%)	Adding more signatures mostly increases the number of entries in the CSR buckets, so bucket lookups become slightly longer.
Samples 1,000 → 2,200 (more_samples)	+120% samples	+2.87s (+107%)	This has a much larger effect because the

Factor changed vs min_clean	Change	runMatcher delta	Explanation
			match kernel scales directly with the number of samples, and the flat sample buffers also take much longer to build.
Sample length 1M → 2M (longer_samples)	+100% length	+2.16s (+81%)	Longer samples mean more positions to check, so the kernel grid becomes much larger. It also doubles the flat buffer construction and host-to-device transfer volume.
Signature length 3K → 10K (longer_signatures)	+233% sig len	+0.17s (+6%)	Signature length has only a small effect because full verification is rarely reached. Most positions are still filtered out by the anchor check before the signature length matters.

- The runtime is driven mostly by **sample count × sample length**, since that determines how many total positions `computeBestMatches` has to examine.
- In contrast, signature count and signature length matter much less. The anchor pre-filter removes almost all positions before full verification, so increasing the number or length of signatures does not increase runtime very much.
- The percentage of `N` characters also has very little impact. Even when a sample contains more `N`s, the hash is still computed normally from the byte values in the window, so the cost of the anchor lookup stays almost the same.
- In other words, the implementation is most sensitive to how much sample data it has to scan, not to how many signatures it stores.

2.3 Kernel Profile

Case	Block	Grid	Occupancy	SM throughput	DRAM throughput	Kernel time
min_clean	256	3,907,000	78.6%	25.5%	2.90%	23.6 ms
high_n	256	3,907,000	80.9%	24.7%	2.83%	23.4 ms
more_signatures	256	3,907,000	77.4%	25.9%	2.94%	23.8 ms
more_samples	256	8,595,400	78.5%	25.5%	2.89%	51.9 ms
longer_samples	256	7,813,000	78.6%	25.6%	2.89%	47.1 ms
longer_signatures	256	3,907,000	78.6%	25.5%	2.90%	23.6 ms

SM throughput of 25% is expected for this anchor-filtering kernel. Most threads only perform the 10-step anchor hash check and exit immediately, so there is not enough work per thread to keep the SMs fully busy.
The compute-versus-memory breakdown, with about 82% compute and only 2.9% DRAM, suggests the kernel is spending far more time on arithmetic from the hash updates than on memory stalls. Occupancy is around 78%, mainly because each thread uses 28 registers.

3. Optimization Analysis

3.1 Optimization 1: Anchor-Based Pre-Filtering

Analysis

Bottleneck:

The original kernel, computeBestMatchesOriginal, is clearly **compute-bound**. SM throughput is already very high at 82–91%, while DRAM throughput stays extremely low at just 0.11–0.21%. This means the GPU is not being held back by memory traffic. The real issue is the amount of computation each thread has to do. Every thread scans through all sample positions, and at each position it performs a full character-by-character comparison against all 700 signatures. Since the signature data is read directly from global memory and compared repeatedly, the kernel ends up doing a massive amount of repeated work.

With 1,000 samples and 700 signatures, there are 700,000 (sample, sig) pairs in total. For each pair, the thread may scan around 1 million positions, and each check can involve up to 3,000 character comparisons. Altogether, this works out to about **2.1 trillion character comparisons** in a single run.

From data/original_suite_nsys.csv, the match kernel takes up about **92–98% of the total GPU time**, so it is clearly the main bottleneck. Using Amdahl's Law with F = 0.95, the theoretical maximum speedup is: 1 / (1 - 0.95) = 20×

So the main opportunity is not in improving memory access, but in reducing how much work the kernel is doing in the first place.

Root cause:

the original algorithm always performs O(positions × signatures × sig_length) work, even for positions that have no chance of matching. In min_clean, almost all of this effort is wasted. About **99.93% of character comparisons** are spent on positions that could never produce a valid match.

Fix:

Reduce the amount of work algorithmically. Instead of brute-forcing every possible position, the final version uses **anchor-based pre-filtering**. For each signature, a 10-character anchor is precomputed on the CPU and stored in a CSR-style hash table with kAnchorKeyCount = 1 << 20 buckets. The kernel is then reorganized so that each thread handles one sample position instead of one (sample, sig) pair.

This changes the work pattern significantly. Each thread first checks the 10-character anchor at its position. Since nearby threads read nearby positions in the same sample, these anchor reads are naturally coalesced. If the hash does not match any populated bucket, the thread exits almost immediately after those 10 steps. Only the small fraction of positions that survive the anchor check go on to full verification, so most threads never load signature characters or enter the expensive comparison loop at all.

Results

Case	Original (s)	Opt 3 (s)	Speedup	Match kernel	GPU time (match, ns)
min_clean	12.92	3.99	**3.24×**	7,862 ms → 23.6 ms	−99.7%
high_n	15.74	3.98	**3.95×**	10,681 ms → 23.4 ms	−99.8%
more_signatures	16.69	4.03	**4.14×**	11,858 ms → 23.8 ms	−99.8%
more_samples	30.07	8.47	**3.55×**	16,743 ms → 51.8 ms	−99.7%
longer_samples	26.54	8.05	**3.30×**	15,878 ms → 47.1 ms	−99.7%
longer_signatures	13.58	3.60	**3.77×**	8,228 ms → 23.6 ms	−99.7%

The ncu profile line up with this explanation. DRAM throughput increased from 0.11% to 2.9% because threads now mostly read short sample windows for the anchor check, instead of repeatedly pulling signature data inside a long inner loop. At the same time, SM throughput fell from 88% to 25%, which is actually expected here, since most threads now do only 10 hash steps and exit instead of spending thousands of iterations on full comparisons. The `__syncthreads()` count also dropped to zero because the new version no longer relies on cooperative tile loading. Altogether, this cuts the match kernel time by about 99.7%, which translates into an overall speedup of around 3.2 to 4.1 times.

3.2 Optimization 2: Host-Side `cudaHostAlloc` Elimination + Hash Kernel Parallelization

Analysis

After Opt 3, the match kernel was no longer the main issue. Its runtime dropped from 7.86s to just 24ms, which is a **99.7% reduction**. Based on `data/optimization_3_suite_nsys.csv`, the runtime for `min_clean` is now dominated by CPU-side work:

Component	Time (s)	% of runMatcher
CPU-side overhead (host work)	3.46	**87%**
computeIntegrityHashes kernel	0.394	10%
H2D transfer	0.108	3%
computeBestMatches kernel	0.024	<1%

The bottleneck shifted elsewhere.

First bottleneck: page pinning from `cudaHostAlloc`. The host code originally allocated `sample_bases` and `sample_qualities`, each around 1 GB, using `cudaHostAlloc`. For buffers this large, pinning memory is expensive. The operating system has to lock a huge number of pages into physical memory, and that setup cost becomes significant. In practice, this explains why CPU overhead grows almost in proportion to sample data size. For example, `more_samples` uses about 2.2 times more data, and the CPU overhead also rises by about 2.25 times.

Fix was to stop using pinned allocation for these two very large buffers and switch them to ordinary `new[]` allocations instead. That avoids paying the heavy page-pinning cost upfront. The tradeoff is that host-to-device transfers become slower, because CUDA now has to stage

pageable memory internally before copying it to the GPU. Even so, the extra transfer cost is much smaller than the time saved from avoiding pinning, so the overall result is still a clear win.

Second bottleneck: the original `computeIntegrityHashes` kernel was too serial. It used one thread per sample, so with 1,000 samples there were only 1,000 threads in total. That is far too little parallelism for a large GPU. On top of that, each thread processed its entire sample sequentially, reading all Phred values one by one and accumulating them in a single register. This meant low occupancy and poor memory behavior, since threads in the same warp were reading from completely different samples rather than nearby memory locations.

Fix was to parallelize the hash computation across each sample. Instead of one thread per sample, the new version launches 256 threads for each sample. Each thread handles every 256th character in that sample, so the work is split evenly across the block. This also makes the memory access pattern much better: neighboring threads now read neighboring bytes, which gives properly coalesced global memory accesses.

Results

Case	Opt 1 (s)	Opt 2 (s)	Speedup	Hash kernel	H2D
min_clean	3.99	**2.68**	1.49×	394 ms → 2.9 ms (−99%)	108 ms → 211 ms (+95 ms)
high_n	3.98	**2.79**	1.43×	394 ms → 2.9 ms (−99%)	108 ms → 212 ms
more_signatures	4.03	**2.78**	1.45×	391 ms → 2.8 ms (−99%)	108 ms → 212 ms
more_samples	8.47	**5.55**	1.53×	400 ms → 5.6 ms (−99%)	232 ms → 462 ms
longer_samples	8.05	**4.84**	1.66×	1,380 ms → 5.9 ms (−99.6%)	211 ms → 429 ms
longer_signatures	3.60	**2.85**	1.26×	393 ms → 2.9 ms (−99%)	106 ms → 212 ms

cudaDeviceSynchronize time, which reflects the total amount of GPU work seen by the CPU, dropped from 526 ms in Opt 3 to just 27 ms in Opt 4 on min_clean. This is a strong sign that both of the remaining bottlenecks were successfully addressed. The biggest improvement appears in longer_samples, with a 1.66× speedup, because that case had the most to gain from the hash-kernel rewrite. Since each sample is twice as long, the old one-thread-per-sample hash kernel took 1.38 s there, so parallelising it in Fix 4B gave the largest absolute reduction.

Overall: Opt 1 + Opt 2 together achieve **4.8–5.5× speedup over original** across all six test cases.

Appendix

F. Figures

Figure 1: Kernel Profile — computeBestMatches (Section 2.3)

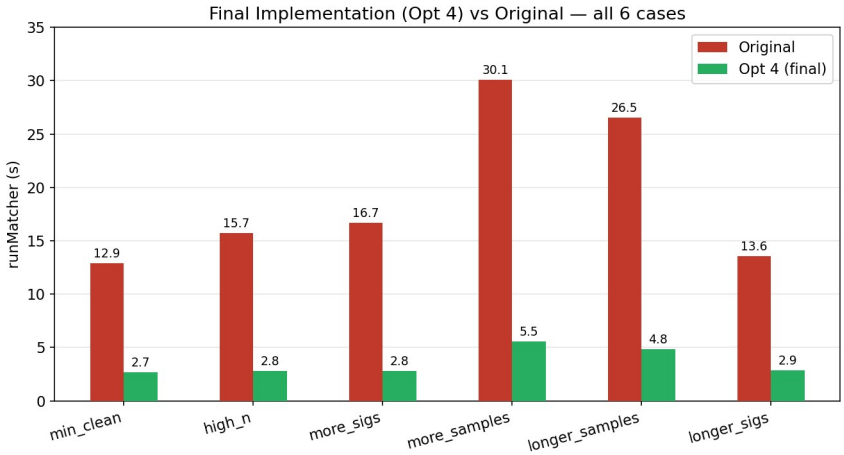


Figure 2: ncu Profile — Optimization 1, Anchor-Based Pre-Filtering (Section 3.1)

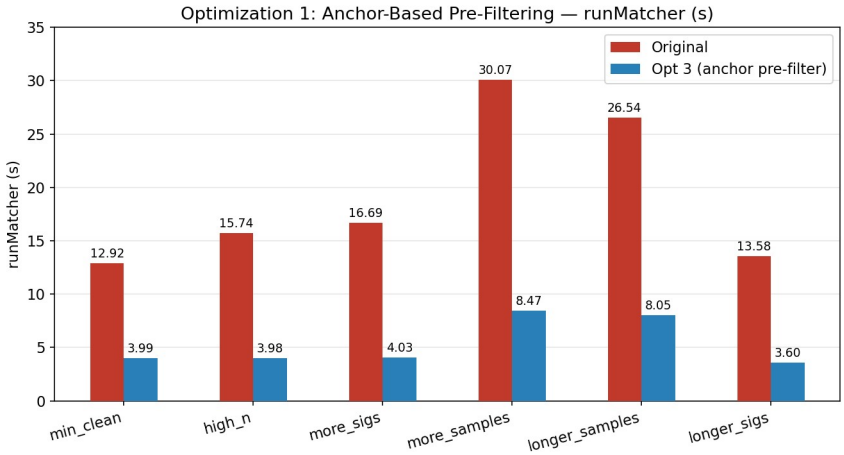
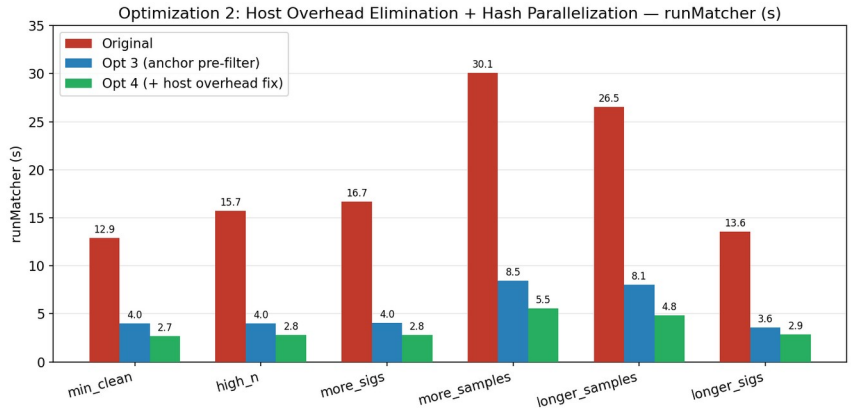


Figure 3: ncu Profile — Optimization 2, Hash Kernel Parallelization (Section 3.2)



A. Reproduction

- **Node**: xgpg (A100-40, SoC GPU cluster, 47 GB PCIe)
- **Build**: make matcher_optimization_4 (flags: -std=c++20 -O3 -lineinfo -dlto -arch=native)
- **Timing**: runMatcher: X.XXX s printed to stderr by common.cc
- **Profiling**: scripts/profile-soc-gpu-report ~/virus-matcher reports/raw/<run> <tag> -- ./matcher_optimization_4 <samp> <sig>

B. Input Generation

Case	Signature generator	Sample generator
min_clean	./gen_sig 700 3000 3000 0.00	./gen_sample sig.fasta 1000 0 1 2 1000000 1000000 10 30 0.00
high_n	./gen_sig 700 3000 3000 0.10	./gen_sample sig.fasta 1000 0 1 2 1000000 1000000 10 30 0.10
more_signatures	./gen_sig 1000 3000 3000 0.00	./gen_sample sig.fasta 1000 0 1 2 1000000 1000000 10 30 0.00
more_samples	./gen_sig 700 3000 3000 0.00	./gen_sample sig.fasta 2200 0 1 2 1000000 1000000 10 30 0.00
longer_samples	./gen_sig 700 3000 3000 0.00	./gen_sample sig.fasta 1000 0 1 2 2000000 2000000 10 30 0.00
longer_signatures	./gen_sig 700 10000 10000 0.00	./gen_sample sig.fasta 1000 0 1 2 1000000 1000000 10 30 0.00

C. Raw Data Files

File	Contents
data/original_suite_runtime.csv	Original kernel: runMatcher, end-to-end, all 6 cases
data/original_suite_ncu.csv	Original kernel: occupancy, SM/DRAM throughput, registers, kernel time
data/original_suite_nsys.csv	Original kernel: H2D, D2H, kernel share of GPU time
data/optimization_3_suite_runtime.csv	Opt 3: runMatcher, end-to-end, all 6 cases
data/optimization_3_suite_ncu.csv	Opt 3: ncu metrics for computeBestMatches
data/optimization_3_suite_nsys.csv	Opt 3: H2D, hash kernel time, match kernel

File	Contents
	time, cudaDeviceSynchronize
data/optimization_3_suite_correctness.csv	Opt 3: diff vs bench-a100-1 and bench-a100-2 (all 6 PASS)
data/optimization_4_suite_runtime.csv	Opt 4: runMatcher, end-to-end, all 6 cases
data/optimization_4_suite_ncu.csv	Opt 4: ncu metrics for computeBestMatches
data/optimization_4_suite_nsys.csv	Opt 4: H2D, hash kernel time, match kernel time, cudaDeviceSynchronize
data/optimization_4_suite_correctness.csv	Opt 4: diff vs bench-a100-1 and bench-a100-2 (all 6 PASS)

D. Full ncu Metrics — Original Kernel

Case	Block	Occupancy	SM%	Compute-mem%	DRAM%	Kernel time (s)
min_clean	256	87.0%	88.3%	40.7%	0.12%	7.87
high_n	256	86.1%	88.4%	39.9%	0.11%	10.65
more_signatures	256	90.8%	85.3%	58.4%	0.16%	11.81
more_samples	256	92.2%	92.0%	42.5%	0.15%	16.72
longer_samples	256	87.1%	88.4%	41.4%	0.21%	15.79
longer_signatures	256	87.4%	82.2%	59.6%	0.18%	8.44

E. Full nsys Metrics — Original vs Opt 3 vs Opt 4 (min_clean)

Metric	Original	Opt 3	Opt 4
runMatcher (s)	12.92	3.99	2.68
Match kernel (ms)	7,862	24	24
Hash kernel (ms)	394	394	2.9
H2D (ms)	202	108	211
D2H (ms)	0.5	0.8	0.8
cudaMalloc (ms)	223	1.2	0.36
cudaDeviceSynchronize (ms)	8,256	526	27